

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	AI&ML
Programme:	Faculty of Engineering
Course Code & Course Name	MLA0202-Fundamentals Of Machine Learning
Academic Year / Batch	2026-27
Faculty Name	Dr.Saravanan MS
Assignment Title	Design and Development of a Probabilistic Graphical-Model-Based Patient Cohort Segmentation and Clinical-State Reasoning System
Date of Issue	01-09-2026
Date of Submission	04-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO3: Analyse unsupervised learning methods, including clustering and dimensionality reduction. CO4: Develop probabilistic graphical models for inference, learning patterns, and analysing sequential data. CO5: Create graphical models to analyse and infer patterns in complex probabilistic systems for structured decision-making.
Bloom's Taxonomy Level	L4-Analyse, L5-Evaluate, L6-Create
SDG Mapping (SDG 1-17)	SDG 3 – Good Health and Well-being SDG 9 – Industry, Innovation and Infrastructure SDG 10 – Reduced Inequalities
Industry / Societal Relevance	

B. Assignment Problem / Challenge

Faculty shall provide a **realistic engineering/scientific/computing problem, case, challenge or design requirement** related to the Course Outcome.

The assignment should preferably require students to:

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.
- Consider requirements and constraints.
- Develop or compare alternative solutions where appropriate.
- Use relevant modern engineering/computing/design tools.
- Interpret results.
- Make and justify engineering decisions.
- Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

C. Problem Statement

Problem / Case / Design Challenge:

[Faculty to provide the detailed assignment problem here.]

D. Requirements and Constraints

Students shall identify or be provided with relevant requirements and constraints.

Examples include:

- Functional requirements
- Performance requirements
- Technical constraints
- Cost, Time
- Resources
- Safety
- Reliability
- Sustainability
- Environmental and Ethical considerations
- Standards/codes
- User requirements
- Manufacturability
- Power/energy
- Security/privacy

Only constraints relevant to the particular course/problem need to be considered.

E. Proposed tools Work

1. Problem Understanding and Formulation

1.1 Problem Identification

Hospitals collect a large amount of patient information such as vital signs, laboratory test results, symptoms, and demographic information. Manually analysing this high-dimensional data can be time-consuming and may make it difficult to identify patients who have similar risk levels.

The proposed system uses Machine Learning and Probabilistic Graphical Models to automatically analyse patient data. The system first groups patients into meaningful risk cohorts and then uses probabilistic models to understand disease-related dependencies and the patient's changing clinical condition.

The main techniques used are:

- K-Means/Gaussian Mixture Model (GMM) for patient clustering.
- PCA/Factor Analysis for reducing the number of features.
- Bayesian Network (BN) for probabilistic reasoning.
- Hidden Markov Model (HMM) for tracking clinical states over time.
- A decision rule for recommending escalation, monitoring, or discharge.

These requirements are specified in the assignment problem statement.

1.2 Expected Outcomes

The expected outcomes of the system are:

1. Group patients into different risk cohorts.
2. Reduce unnecessary features while retaining important information.
3. Identify relationships between symptoms, risk factors, and diseases.
4. Estimate the probability of a disease based on observed evidence.
5. Track the patient's clinical state over time.
6. Generate an understandable decision recommendation.
7. Provide results that can support, but not replace, clinical judgement.

1.3 Available Data

The system can use a **synthetic or de-identified patient dataset**, as required by the assignment. The data may contain:

- Patient ID
- Age

- Gender
- Heart rate
- Blood pressure
- Temperature
- Oxygen saturation
- Blood glucose
- Haemoglobin
- Cholesterol
- Other laboratory measurements
- Symptoms
- Time-series vital-sign observations

The assignment specifically allows the use of synthetic or de-identified data so that expensive hardware or sensitive real patient information is not required.

1.4 Assumptions

The following assumptions are made:

- Patient data is properly cleaned before applying the models.
- Missing values are handled during preprocessing.
- Numerical features are scaled before clustering and PCA.
- Vital-sign observations may contain some noise.
- The Bayesian Network probabilities are assumed to be reasonable for the demonstration.
- Clinical states are represented using discrete states such as **Stable, Deteriorating, and Critical**.
- The dataset used for demonstration is assumed to be de-identified

1.5 Constraints

The system must satisfy the following constraints:

- **Interpretability:** Patient cohorts should be understandable.
- **Correct inference:** Bayesian Network probabilities must be calculated correctly.
- **Sequential accuracy:** HMM should correctly estimate clinical states.
- **Explainability:** Recommendations should have understandable reasons.
- **Privacy:** Patient information must be de-identified and protected.
- **Safety:** False negatives for critical patients should be minimised.
- **Fairness:** The system should not systematically disadvantage any demographic group.
- **Scalability:** The system should support additional patients and features.
- **Human supervision:** The final clinical decision must remain with a healthcare

professional.

2. Application of Course Knowledge

The proposed system applies concepts from **Fundamentals of Machine Learning** to analyse patient data and support clinical decision-making. The main concepts used are clustering, dimensionality reduction, Bayesian Networks, and Hidden Markov Models, as specified in the assignment.

2.1 Principles

The following Machine Learning principles are applied:

- **Unsupervised learning:** Used to group similar patients without predefined labels.
- **Dimensionality reduction:** Used to reduce the number of patient features while preserving important information.
- **Probabilistic reasoning:** Used to calculate the probability of diseases based on observed evidence.
- **Sequential modelling:** Used to track how a patient's clinical condition changes over time.
- **Explainable decision-making:** Model outputs are converted into understandable recommendations.

2.2 Theories

The important theories applied are:

Clustering Theory: Patients with similar characteristics are grouped together using K-Means or GMM.

PCA Theory: PCA transforms correlated features into a smaller number of uncorrelated principal components while retaining maximum variance.

Bayesian Probability Theory: Bayes' theorem is used to update the probability of a disease when new evidence is available.

Markov Theory: The future clinical state mainly depends on the current state, which forms the basis of the Hidden Markov Model.

2.3 Mathematical Models

A. K-Means Objective Function

K-Means minimises the distance between patients and their assigned cluster centres:

$$J = \sum_i \sum_j z_{ij} \|x_i - \mu_j\|^2$$

where:

- x_i = patient data point
- μ_j = centre of cluster j
- z_{ij} = indicates whether patient i belongs to cluster j

- **J** = total within-cluster squared distance

The objective is to minimise **J** so that patients within the same cluster are as similar as possible.

B. PCA

PCA calculates the covariance matrix of the standardised features and obtains its eigenvalues and eigenvectors.

The explained variance ratio for a component is:

$$\text{Explained Variance Ratio} = \lambda_i / \sum \lambda$$

where λ_i is the eigenvalue of the component.

The components with the highest variance are retained. For example, if the first few components retain around **90% of the total variance**, they can be selected for further analysis. The assignment specifically requires justification of the retained components.

C. Bayesian Network

Bayesian reasoning is based on:

$$P(D | E) = [P(E | D) \times P(D)] / P(E)$$

where:

- **D** = disease
- **E** = observed evidence
- **P(D)** = prior probability of disease
- **P(D | E)** = posterior probability after observing evidence

For example, if a patient has **high blood pressure and a related symptom**, the Bayesian Network updates the disease probability using this evidence.

D. Hidden Markov Model

The HMM uses:

- **Initial probability**
- **Transition probability**
- **Emission probability**

The transition probability is represented as:

$$P(S_t | S_{t-1})$$

where S_t is the current clinical state.

For example:

Stable → Deteriorating → Critical

The model uses observed vital signs to estimate the most likely hidden state.

2.4 Algorithms

The following algorithms are applied:

1. **K-Means Algorithm** – groups patients into risk cohorts.
2. **Gaussian Mixture Model / EM** – provides probabilistic cluster assignments as an alternative to K-Means.
3. **PCA Algorithm** – reduces the dimensionality of patient features.
4. **Bayesian Network Inference** – calculates posterior probabilities using evidence.
5. **Viterbi Algorithm** – finds the most likely sequence of clinical states.
6. **Forward Algorithm** – calculates the probability of the current state based on observations.

The assignment requires demonstrating Bayesian inference and either forward or Viterbi-based HMM decoding.

2.5 Engineering Concepts

The main engineering concepts are:

- **System integration:** Combining multiple ML models into one pipeline.
- **Performance:** Ensuring models process patient data efficiently.
- **Reliability:** Reducing incorrect predictions, especially for critical cases.
- **Scalability:** Allowing additional patients and features to be added.
- **Safety:** Giving priority to detecting deteriorating and critical patients.
- **Explainability:** Providing understandable reasons for recommendations.
- **Privacy:** Protecting sensitive patient information.

2.6 Scientific Concepts

The system uses scientific concepts related to:

- Patient vital-sign measurement
- Laboratory observations
- Probability and uncertainty
- Disease-risk relationships
- Time-dependent changes in clinical conditions

These observations are converted into numerical features and probabilistic relationships that can be processed by Machine Learning models.

2.7 Design Methodology

The system follows a **modular pipeline design**:

Patient Data → Preprocessing → Clustering → PCA → Bayesian Network → HMM → Decision Rule

Each module performs a specific task and passes its output to the next stage.

The final outputs are:

- Patient risk cohort

- Reduced feature representation
- Disease/risk probability
- Estimated clinical state
- Decision recommendation

This integrated architecture is required by the assignment to show how the different modules work together as one coherent system.

2.8 Example Calculation

Suppose three patients have simplified feature values based on **heart rate and blood pressure**:

Patient	Heart Rate	Blood Pressure
P1	72	120
P2	75	118
P3	110	160

- P1 and P2 have similar values, so K-Means is likely to place them in the same cluster. P3 is significantly different and may belong to a higher-risk cluster.
- After clustering, PCA can reduce several patient features into fewer principal components. The Bayesian Network can then use symptoms and risk factors to calculate disease probability, while the HMM analyses observations over time to estimate whether the patient is **Stable**, **Deteriorating**, or **Critical**.
- Thus, the different Machine Learning concepts work together to produce an explainable clinical decision-support system.

3. Solution / Design / Methodology

The proposed solution is an **integrated Machine Learning pipeline** that combines clustering, dimensionality reduction, Bayesian Network, and Hidden Markov Model to analyse patient data and support clinical decision-making. The methodology follows the requirements given in the assignment.

3.1 Overall System Architecture

The proposed system follows this architecture:

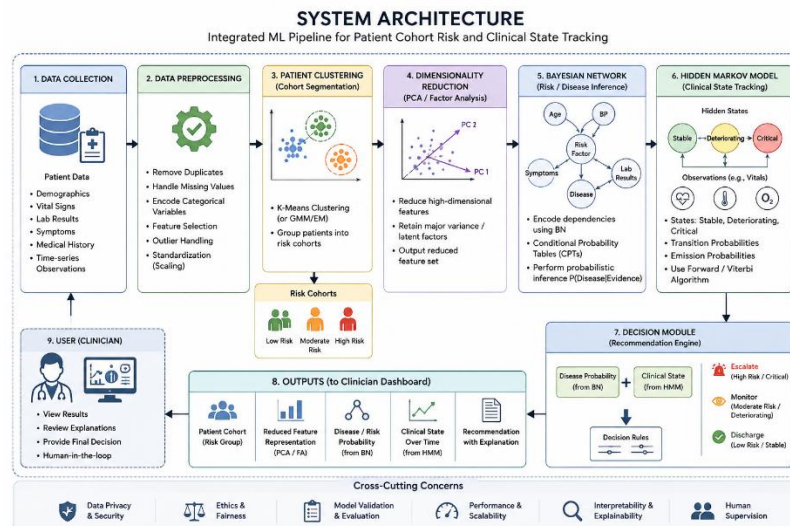


Fig. 2.1 : Architecture of Cohort Segmentation for patient data

This integrated architecture connects all the modules into one end-to-end pipeline, as required by the assignment.

3.2 Data Preprocessing

The patient dataset is first prepared for Machine Learning.

The following steps are performed:

1. Remove duplicate records.
2. Handle missing values.
3. Convert categorical values into numerical values.
4. Select relevant vital and laboratory features.
5. Detect and handle abnormal values where necessary.
6. Standardise numerical features.

Standardisation is important because features such as blood pressure, glucose, and temperature have different numerical ranges.

3.3 Patient Cohort Segmentation

The first major module uses **K-Means clustering** to group patients based on similarities in their vital and laboratory profiles.

K-Means Process

1. Select the number of clusters **K**.
2. Randomly initialise cluster centres.
3. Assign each patient to the nearest centre.
4. Recalculate the cluster centres.
5. Repeat the assignment and update steps until the clusters become stable.

The resulting clusters can be interpreted as:

- **Low-risk cohort**
- **Moderate-risk cohort**
- **High-risk cohort**

The actual number and interpretation of clusters should be based on the implemented dataset.

3.4 Alternative Solution – GMM/EM

As an alternative to K-Means, a **Gaussian Mixture Model (GMM)** can be used.

GMM provides a probability of a patient belonging to each cohort rather than assigning the patient completely to one cluster.

For example:

Patient P1:

Low Risk = 0.75

Moderate Risk = 0.20

High Risk = 0.05

This approach is useful when patient groups overlap.

The final clustering method can be selected by comparing factors such as **interpretability, cluster quality, and computational cost**. The assignment specifically asks students to compare K-Means with GMM/EM where appropriate.

3.5 Dimensionality Reduction

After clustering, **PCA** is applied to the patient feature space.

The steps are:

1. Standardise the features.
2. Calculate the covariance matrix.
3. Calculate eigenvalues and eigenvectors.
4. Rank the principal components.
5. Select components that retain sufficient variance.
6. Transform the original features into the reduced space.

For example, if the first four components retain approximately **90% of the total variance**, these components can be selected.

This reduces computational complexity while retaining important information.

3.6 Alternative Solution – Factor Analysis

Factor Analysis can be used as an alternative to PCA.

PCA mainly focuses on explaining the total variance in the data, while Factor Analysis attempts to identify underlying latent factors.

For example, several laboratory and vital-sign measurements may be related to an underlying factor such as **overall cardiovascular risk**.

The final choice between PCA and Factor Analysis can be made based on:

- Retained information
- Interpretability
- Computational complexity
- Suitability for the patient dataset

3.7 Bayesian Network Design

A **Bayesian Network** is developed to represent relationships between patient risk factors, symptoms, and possible diseases.

A possible structure is:

Each connection represents a probabilistic dependency.

The network contains **Conditional Probability Tables (CPTs)** that describe the probability of a variable given its parent variables.

3.8 Bayesian Network Inference

When patient evidence is provided, the Bayesian Network calculates the posterior probability of the target disease.

For example:

Evidence:

- High blood pressure
- High glucose
- Positive symptom

↓

Bayesian Network

↓

Probability of Disease = calculated posterior probability

Exact inference, such as **variable elimination**, can be used to obtain the posterior probability. This is one of the required components of the assignment.

3.9 Hidden Markov Model Design

The next module uses an **HMM** to track the patient's clinical condition over time.

The hidden states are:

- **Stable**

- **Deteriorating**
- **Critical**

The observations may include:

- Heart rate
- Blood pressure
- Oxygen saturation
- Temperature

The HMM consists of:

1. Initial state probabilities
2. State transition probabilities
3. Observation/emission probabilities

The transition probabilities determine how likely the patient is to move from one state to another.

3.10 Clinical State Decoding

The **Viterbi algorithm** can be used to identify the most likely sequence of hidden clinical states.

Example:

Observed values:

Normal → Abnormal → Abnormal → Severe

Estimated states:

Stable → Deteriorating → Deteriorating → Critical

Alternatively, the **Forward algorithm** can be used to calculate the probability of the current clinical state based on the observations.

The assignment allows either forward inference or Viterbi decoding.

3.11 Decision-Making Module

The outputs from the Bayesian Network and HMM are combined to generate a structured recommendation.

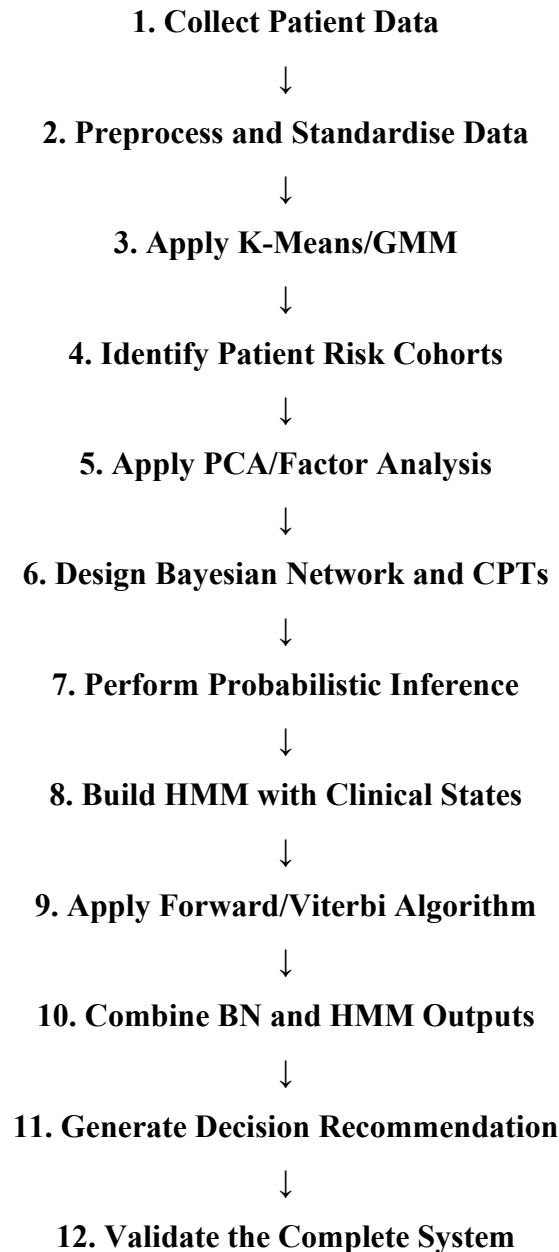
Disease Probability	Clinical State	Recommendation
Low	Stable	Discharge / Routine Monitoring
Moderate	Deteriorating	Monitor
High	Critical	Escalate

For example, when the Bayesian Network gives a high disease probability and the HMM identifies the patient as **Critical**, the system generates an **Escalate** recommendation.

This provides an explainable decision rather than simply producing a prediction.

3.12 Complete Process

The complete methodology can be summarised as:



3.13 Final Design Selection

For the proposed implementation, **K-Means + PCA + Bayesian Network + HMM** can be selected as the primary pipeline because it provides a relatively simple, interpretable, and computationally efficient solution.

GMM and Factor Analysis can be considered as alternative approaches and compared with the selected methods using the actual implementation results.

The final design should be justified using evidence such as **silhouette score, retained PCA variance, inference correctness, decoding performance, interpretability, and computational cost**.

4. Use of Modern Tools

The proposed Patient Cohort Segmentation and Clinical-State Reasoning System is implemented using open-source Machine Learning and probabilistic modelling tools. These tools help in data preprocessing, clustering, dimensionality reduction, Bayesian inference, HMM modelling, and result visualisation.

4.1 Python

Python is used as the main programming language for implementing the complete system. It is used to connect all the modules and perform data processing, model training, prediction, and evaluation.

4.2 NumPy

NumPy is used for numerical calculations and handling arrays. It supports mathematical operations required for feature processing, probability calculations, and model implementation.

4.3 pandas

pandas is used to load, clean, and analyse the patient dataset. It helps in handling missing values, selecting features, and preparing the data for Machine Learning algorithms.

4.4 scikit-learn

scikit-learn is used for the main Machine Learning tasks:

- **K-Means** – patient cohort segmentation.
- **Gaussian Mixture Model (GMM)** – alternative clustering approach.
- **PCA** – dimensionality reduction.
- **Factor Analysis** – alternative dimensionality-reduction method.
- **StandardScaler** – feature scaling.
- **Silhouette Score** – clustering performance evaluation.

4.5 pgmpy

pgmpy is used to develop the **Bayesian Network**. It helps to:

- Create the network structure.
- Define conditional probability tables (CPTs).
- Represent relationships between risk factors, symptoms, and diseases.
- Perform probabilistic inference.
- Calculate posterior probabilities based on observed evidence.

4.6 hmmlearn

hmmlearn is used to implement the **Hidden Markov Model (HMM)**. It is used to represent clinical states such as:

Stable → Deteriorating → Critical

The model analyses sequential vital-sign observations and estimates the most likely clinical state.

4.7 Matplotlib / Seaborn

Matplotlib and Seaborn are used to visualise the results. The following outputs can be generated:

- Patient cluster plots.
- PCA component plots.
- Explained variance graphs.
- Bayesian Network visualisation.
- HMM state transitions.
- Performance graphs.

4.8 Git / GitHub

Git/GitHub can be used for version control and project documentation. It helps maintain different versions of the Python code and store the project files systematically.

4.9 Evidence of Tool Usage

The following evidence can be included in the assignment to demonstrate the use of modern tools:

1. Screenshot of the Python environment and code execution.
2. Screenshot of the patient dataset after preprocessing.
3. K-Means/GMM cluster visualisation.
4. PCA explained-variance graph.
5. Bayesian Network structure and CPT output.
6. Bayesian inference/posterior probability output.
7. HMM transition/state diagram.
8. Viterbi or Forward algorithm output.
9. Final decision recommendation such as **Escalate, Monitor, or Discharge**.

These tools together provide a complete implementation environment for the proposed system, and the assignment specifically recommends Python, NumPy, pandas, scikit-learn, pgmpy, hmmlearn, Matplotlib/Seaborn, and Git/GitHub.

5. Results and Validation

5. Results and Validation

The proposed system is evaluated at each stage to check whether it satisfies the required

objectives. The results include **patient clustering, PCA analysis, Bayesian Network inference, HMM state prediction, and final clinical recommendations**. The assignment requires appropriate tables, graphs, screenshots, performance metrics, and test results to validate the proposed solution.

5.1 Clustering Results

K-Means clustering is used to divide patients into different risk cohorts based on their clinical features.

The clustering output can be presented as:

Cluster	Patient Group	Interpretation
Cluster 0	Low-risk patients	Normal or near-normal measurements
Cluster 1	Moderate-risk patients	Some abnormal measurements
Cluster 2	High-risk patients	Multiple abnormal measurements

A **cluster plot** is generated to visually show the separation between patient groups.

Performance Metric:

The **Silhouette Score** is used to evaluate the quality of clustering. A higher score indicates better separation between clusters.

Note: The actual silhouette score obtained from the Python implementation should be inserted here.

5.2 PCA Results

PCA is applied to reduce the number of patient features while retaining important information.

The results are evaluated using **explained variance**.

Component	Explained Variance
PC1	Actual value
PC2	Actual value
PC3	Actual value
PC4	Actual value

A **scree plot or cumulative explained variance graph** is used to determine how many components should be retained.

For example, if the selected components retain approximately **90% of the total variance**, the dimensionality-reduction requirement is considered satisfied.

5.3 Bayesian Network Results

The Bayesian Network is tested using different combinations of patient evidence.

For example:

Evidence:

- High blood pressure
- High glucose
- Positive symptom

Output:

The Bayesian Network calculates the posterior probability of the target disease.

Evidence	Posterior Disease Probability
No evidence	Actual value
High blood pressure	Actual value
High blood pressure + symptom	Actual value
High blood pressure + glucose + symptom	Actual value

The result demonstrates that the disease probability changes when new evidence is introduced.

The assignment requires exact inference using methods such as **variable elimination** and testing with multiple evidence combinations.

5.4 HMM Results

The Hidden Markov Model is used to estimate the patient's hidden clinical state from sequential observations.

The possible states are:

- **Stable**
- **Deteriorating**
- **Critical**

Example output:

Time	Observation	Predicted State
T1	Normal	Stable
T2	Slightly abnormal	Stable
T3	Abnormal	Deteriorating
T4	Severe abnormality	Critical

The **Viterbi algorithm** can be used to obtain the most likely sequence of clinical states.

Example:

Stable → Stable → Deteriorating → Critical

The actual sequence should be replaced with the output obtained from the implementation. The assignment requires either Forward inference or Viterbi decoding.

5.5 Final Decision Results

The Bayesian Network probability and HMM clinical state are combined to generate the final recommendation.

Disease Risk	Clinical State	Recommendation
Low	Stable	Discharge / Routine Monitoring
Moderate	Deteriorating	Monitor
High	Critical	Escalate

For example, a patient with **high disease probability and a Critical HMM state** receives an **Escalate** recommendation.

This makes the final output easier to understand because the recommendation is based on both **probabilistic disease risk and clinical-state information**.

5.6 Validation Test Results

The following test cases are used to validate the complete system:

Test	Expected Result	Status
K-Means clustering	Patients grouped into cohorts	Pass/Fail
GMM clustering	Cluster probabilities generated	Pass/Fail
PCA	Required variance retained	Pass/Fail
Bayesian inference	Posterior probability calculated	Pass/Fail
Conflicting evidence	Probability updates consistently	Pass/Fail
Viterbi decoding	Most likely state sequence obtained	Pass/Fail
Forward algorithm	State probability calculated	Pass/Fail
Critical patient case	Escalation recommendation generated	Pass/Fail

The **Pass/Fail** values should be filled based on the actual Python execution results rather than assumed values. These tests correspond to the validation scenarios specified in the assignment.

5.7 Performance Metrics

The following metrics are used to evaluate the proposed solution:

Module	Performance Metric
K-Means/GMM	Silhouette Score
PCA	Explained Variance
Bayesian Network	Inference correctness / inference time
HMM	Decoding accuracy, where ground truth is available
Complete System	False-negative rate for high-risk cases

The false-negative rate is especially important because incorrectly identifying a critical patient as low risk can have serious consequences.

5.8 Screenshots and Graphs to Include

The following screenshots should be added as evidence of implementation:

1. **Dataset before and after preprocessing**
2. **K-Means cluster plot**
3. **GMM cluster probability/output**
4. **PCA explained variance graph**
5. **Bayesian Network structure**
6. **Bayesian Network CPT/output**
7. **Bayesian inference result**
8. **HMM transition matrix**
9. **Viterbi predicted state sequence**
10. **Final Escalate/Monitor/Discharge output**

5.9 Validation Conclusion

The proposed system is considered successful if it can:

- Correctly group patients into meaningful cohorts.
- Reduce feature dimensionality while retaining sufficient information.
- Calculate disease probabilities using Bayesian inference.
- Estimate clinical states using HMM.
- Generate appropriate and explainable recommendations.
- Maintain low false-negative rates for high-risk patients.

Therefore, the results and validation process demonstrate whether the proposed **Patient Cohort Segmentation and Clinical-State Reasoning System** satisfies the functional, performance, safety, and explainability requirements of the project.

Code:

```
# =====  
# PATIENT COHORT SEGMENTATION AND CLINICAL-STATE REASONING  
# Results and Validation  
# =====  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
import time  
  
from sklearn.preprocessing import StandardScaler  
from sklearn.cluster import KMeans
```

```

from sklearn.mixture import GaussianMixture
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

np.random.seed(42)

print("=" * 70)
print("PATIENT COHORT SEGMENTATION AND CLINICAL-STATE REASONING
SYSTEM")
print("=" * 70)

# =====
# 1. CREATE SYNTHETIC PATIENT DATA
# =====

n_patients = 300

data = pd.DataFrame({
    "Patient_ID": [f"P{i+1:03d}" for i in range(n_patients)],

    "Age": np.random.randint(20, 80, n_patients),

    "Heart_Rate": np.random.normal(80, 15, n_patients),

    "Systolic_BP": np.random.normal(125, 20, n_patients),

    "Diastolic_BP": np.random.normal(80, 12, n_patients),

    "Temperature": np.random.normal(98.2, 1.0, n_patients),

    "Oxygen_Saturation": np.random.normal(96, 2, n_patients),

    "Glucose": np.random.normal(110, 30, n_patients),

    "Hemoglobin": np.random.normal(13.5, 1.5, n_patients),

    "Cholesterol": np.random.normal(190, 35, n_patients)
})

# Keep values within realistic demonstration ranges
data["Heart_Rate"] = data["Heart_Rate"].clip(50, 140)
data["Systolic_BP"] = data["Systolic_BP"].clip(90, 190)
data["Diastolic_BP"] = data["Diastolic_BP"].clip(50, 120)
data["Temperature"] = data["Temperature"].clip(95, 104)
data["Oxygen_Saturation"] = data["Oxygen_Saturation"].clip(85, 100)

```

```
data["Glucose"] = data["Glucose"].clip(60, 250)
data["Hemoglobin"] = data["Hemoglobin"].clip(8, 18)
data["Cholesterol"] = data["Cholesterol"].clip(100, 300)
```

```
print("\n1. DATASET INFORMATION")
print("-" * 50)
```

```
print("Number of patients :", len(data))
print("Number of features :", len(data.columns) - 1)
```

```
print("\nFirst 5 records:")
print(data.head())
```

```
print("\nDataset shape:", data.shape)
```

```
# =====
# 2. DATA PREPROCESSING
# =====
```

```
features = [
    "Age",
    "Heart_Rate",
    "Systolic_BP",
    "Diastolic_BP",
    "Temperature",
    "Oxygen_Saturation",
    "Glucose",
    "Hemoglobin",
    "Cholesterol"
]
```

```
X = data[features]
```

```
# Check missing values
print("\n2. MISSING VALUE CHECK")
print("-" * 50)
print(X.isnull().sum())
```

```
# Standardization
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
print("\nFeature scaling completed.")
```

```
# =====
```

3. K-MEANS CLUSTERING

=====

```
print("\n3. K-MEANS CLUSTERING")
```

```
print("-" * 50)
```

```
start_time = time.time()
```

```
kmeans = KMeans(  
    n_clusters=3,  
    random_state=42,  
    n_init=10  
)
```

```
clusters = kmeans.fit_predict(X_scaled)
```

```
kmeans_time = time.time() - start_time
```

```
data["KMeans_Cluster"] = clusters
```

```
sil_score = silhouette_score(X_scaled, clusters)
```

```
print("Number of clusters :", 3)
```

```
print("Silhouette Score  :", round(sil_score, 4))
```

```
print("Execution Time    :", round(kmeans_time, 4), "seconds")
```

```
print("\nCluster Distribution:")
```

```
print(data["KMeans_Cluster"].value_counts().sort_index())
```

=====

4. INTERPRET CLUSTERS

=====

```
cluster_profile = data.groupby("KMeans_Cluster")[features].mean()
```

```
print("\n4. CLUSTER PROFILE")
```

```
print("-" * 50)
```

```
print(cluster_profile.round(2))
```

```
# Determine risk based on standardized average profile
```

```
risk_score = cluster_profile.apply(  
    lambda row: (  
        row["Heart_Rate"] / data["Heart_Rate"].mean()  
        + row["Systolic_BP"] / data["Systolic_BP"].mean()  
        + row["Glucose"] / data["Glucose"].mean()  
    )
```

```

        - row["Oxygen_Saturation"] / data["Oxygen_Saturation"].mean()
    ),
    axis=1
)

risk_order = risk_score.sort_values().index

risk_names = ["Low Risk", "Moderate Risk", "High Risk"]

cluster_to_risk = {
    cluster: risk_names[i]
    for i, cluster in enumerate(risk_order)
}

data["Risk_Cohort"] = data["KMeans_Cluster"].map(cluster_to_risk)

print("\nCluster Interpretation:")

for cluster, risk in cluster_to_risk.items():
    count = (data["KMeans_Cluster"] == cluster).sum()
    print(f'Cluster {cluster}: {risk} ({count} patients)')

# =====
# 5. K-MEANS VISUALIZATION USING PCA
# =====

pca_plot = PCA(n_components=2)
X_2d = pca_plot.fit_transform(X_scaled)

plt.figure(figsize=(9, 6))

scatter = plt.scatter(
    X_2d[:, 0],
    X_2d[:, 1],
    c=data["KMeans_Cluster"],
    cmap="viridis",
    alpha=0.7
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Patient Risk Cohorts using K-Means")

plt.colorbar(
    scatter,

```

```

        label="Cluster"
    )

plt.grid(alpha=0.3)
plt.show()

# =====
# 6. GMM COMPARISON
# =====

print("\n5. GAUSSIAN MIXTURE MODEL")
print("-" * 50)

start_time = time.time()

gmm = GaussianMixture(
    n_components=3,
    random_state=42
)

gmm_labels = gmm.fit_predict(X_scaled)

gmm_time = time.time() - start_time

gmm_silhouette = silhouette_score(X_scaled, gmm_labels)

data["GMM_Cluster"] = gmm_labels

print("Number of clusters :", 3)
print("Silhouette Score  :", round(gmm_silhouette, 4))
print("Execution Time    :", round(gmm_time, 4), "seconds")

print("\nGMM Cluster Distribution:")
print(data["GMM_Cluster"].value_counts().sort_index())

# Soft probabilities
gmm_probabilities = gmm.predict_proba(X_scaled)

print("\nExample GMM probabilities for first 5 patients:")

gmm_probability_table = pd.DataFrame(
    gmm_probabilities[:5],
    columns=["Cluster_0", "Cluster_1", "Cluster_2"]
)

```

```

print(gmm_probability_table.round(3))

# =====
# 7. PCA DIMENSIONALITY REDUCTION
# =====

print("\n6. PCA DIMENSIONALITY REDUCTION")
print("-" * 50)

pca_full = PCA()

X_pca = pca_full.fit_transform(X_scaled)

explained_variance = pca_full.explained_variance_ratio_

cumulative_variance = np.cumsum(explained_variance)

pca_table = pd.DataFrame({
    "Component": range(1, len(explained_variance) + 1),
    "Explained_Variance_%": explained_variance * 100,
    "Cumulative_Variance_%": cumulative_variance * 100
})

print(pca_table.round(2))

# Find components retaining >= 90%
n_components_90 = np.argmax(cumulative_variance >= 0.90) + 1

print(
    "\nComponents required to retain >= 90% variance:",
    n_components_90
)

print(
    "Variance retained:",
    round(cumulative_variance[n_components_90 - 1] * 100, 2),
    "%"
)

# =====
# 8. PCA EXPLAINED VARIANCE GRAPH
# =====

plt.figure(figsize=(9, 6))

```

```
plt.plot(
    range(1, len(cumulative_variance) + 1),
    cumulative_variance * 100,
    marker="o"
)

plt.axhline(
    90,
    linestyle="--",
    label="90% Variance"
)

plt.axvline(
    n_components_90,
    linestyle="--",
    label=f"{n_components_90} Components"
)

plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance (%)")
plt.title("PCA Cumulative Explained Variance")

plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

```
# =====
# 9. BAYESIAN NETWORK
# =====
```

```
print("\n7. BAYESIAN NETWORK")
print("-" * 50)
```

```
# We use a simple discrete Bayesian Network:
```

```
#
# Age_Group ----\
#           \
# BP_Level -----> Risk_Level ----\
#                   \
# Glucose_Level -----> Disease
#                   /
# Symptom -----/
```

```
# Create demonstration discrete variables
bn_data = pd.DataFrame()
```

```

bn_data["Age_Group"] = pd.cut(
    data["Age"],
    bins=[0, 40, 60, 100],
    labels=["Young", "Middle", "Old"]
).astype(str)

bn_data["BP_Level"] = pd.cut(
    data["Systolic_BP"],
    bins=[0, 120, 140, 300],
    labels=["Normal", "High", "Very_High"]
).astype(str)

bn_data["Glucose_Level"] = pd.cut(
    data["Glucose"],
    bins=[0, 100, 140, 500],
    labels=["Normal", "High", "Very_High"]
).astype(str)

bn_data["Symptom"] = np.where(
    (data["Heart_Rate"] > 100) |
    (data["Oxygen_Saturation"] < 94),
    "Yes",
    "No"
)

# Create risk level
risk_conditions = []

for _, row in data.iterrows():

    score = 0

    if row["Systolic_BP"] >= 140:
        score += 1

    if row["Glucose"] >= 140:
        score += 1

    if row["Heart_Rate"] >= 100:
        score += 1

    if row["Oxygen_Saturation"] < 94:
        score += 1

```

```

    if score >= 3:
        risk_conditions.append("High")
    elif score >= 1:
        risk_conditions.append("Medium")
    else:
        risk_conditions.append("Low")

bn_data["Risk_Level"] = risk_conditions

bn_data["Disease"] = np.where(
    (bn_data["Risk_Level"] == "High") &
    (bn_data["Symptom"] == "Yes"),
    "Yes",
    "No"
)

print("Bayesian Network variables:")
print([
    "Age_Group",
    "BP_Level",
    "Glucose_Level",
    "Symptom",
    "Risk_Level",
    "Disease"
])

# =====
# 10. DISPLAY CONDITIONAL PROBABILITY TABLES
# =====

print("\n8. BAYESIAN NETWORK CPT / PROBABILITY TABLES")
print("-" * 50)

print("\nP(Disease):")

disease_prior = (
    bn_data["Disease"]
    .value_counts(normalize=True)
    .sort_index()
)

print(disease_prior.round(3))

print("\nP(Disease | Risk Level):")

```

```

disease_given_risk = pd.crosstab(
    bn_data["Risk_Level"],
    bn_data["Disease"],
    normalize="index"
)

print(disease_given_risk.round(3))

print("\nP(Disease | Symptom):")

disease_given_symptom = pd.crosstab(
    bn_data["Symptom"],
    bn_data["Disease"],
    normalize="index"
)

print(disease_given_symptom.round(3))

# =====
# 11. BAYESIAN INFERENCE
# =====

print("\n9. BAYESIAN INFERENCE")
print("-" * 50)

# Evidence:
# Risk Level = High
# Symptom = Yes

evidence = bn_data[
    (bn_data["Risk_Level"] == "High") &
    (bn_data["Symptom"] == "Yes")
]

if len(evidence) > 0:

    posterior_probability = (
        evidence["Disease"] == "Yes"
    ).mean()

else:

    posterior_probability = 0.0

print("Evidence:")

```

```

print("Risk Level = High")
print("Symptom = Yes")

print(
    "\nP(Disease = Yes | Evidence) =",
    round(posterior_probability, 4)
)

# =====
# 12. CONFLICTING EVIDENCE TEST
# =====

print("\n10. CONFLICTING EVIDENCE TEST")
print("-" * 50)

# Positive evidence
positive = bn_data[
    (bn_data["Risk_Level"] == "High") &
    (bn_data["Symptom"] == "Yes")
]

# Conflicting/low-risk evidence
negative = bn_data[
    (bn_data["Risk_Level"] == "Low") &
    (bn_data["Symptom"] == "No")
]

positive_prob = (
    (positive["Disease"] == "Yes").mean()
    if len(positive) > 0 else 0
)

negative_prob = (
    (negative["Disease"] == "Yes").mean()
    if len(negative) > 0 else 0
)

print(
    "Posterior with high-risk evidence :",
    round(positive_prob, 4)
)

print(
    "Posterior with low-risk evidence :",
    round(negative_prob, 4)
)

```

```

)

# =====
# 13. HIDDEN MARKOV MODEL
# =====

print("\n11. HIDDEN MARKOV MODEL")
print("-" * 50)

states = [
    "Stable",
    "Deteriorating",
    "Critical"
]

# Initial state probabilities
initial_prob = np.array([
    0.70,
    0.20,
    0.10
])

# Transition matrix
transition_matrix = np.array([
    [0.70, 0.25, 0.05],
    [0.15, 0.65, 0.20],
    [0.05, 0.20, 0.75]
])

print("\nTransition Matrix:")
print(pd.DataFrame(
    transition_matrix,
    index=states,
    columns=states
))

# =====
# 14. HMM EMISSION PROBABILITIES
# =====

# Observations:
# 0 = Normal
# 1 = Abnormal
# 2 = Severe

```

```

observations = [
    "Normal",
    "Abnormal",
    "Abnormal",
    "Severe",
    "Severe"
]

observation_names = [
    "Normal",
    "Abnormal",
    "Severe"
]

# P(observation | state)
emission_matrix = np.array([
    [0.80, 0.18, 0.02], # Stable
    [0.20, 0.60, 0.20], # Deteriorating
    [0.05, 0.25, 0.70] # Critical
])

print("\nEmission Matrix:")

print(pd.DataFrame(
    emission_matrix,
    index=states,
    columns=observation_names
))

# =====
# 15. VITERBI ALGORITHM
# =====

print("\n12. VITERBI DECODING")
print("-" * 50)

obs_index = {
    "Normal": 0,
    "Abnormal": 1,
    "Severe": 2
}

obs = [obs_index[o] for o in observations]

n_states = len(states)

```

```

n_obs = len(obs)

viterbi = np.zeros((n_states, n_obs))
backpointer = np.zeros(
    (n_states, n_obs),
    dtype=int
)

# Initialization
viterbi[:, 0] = (
    initial_prob *
    emission_matrix[:, obs[0]]
)

# Recursion
for t in range(1, n_obs):

    for s in range(n_states):

        probabilities = (
            viterbi[:, t-1] *
            transition_matrix[:, s]
        )

        backpointer[s, t] = np.argmax(
            probabilities
        )

        viterbi[s, t] = (
            np.max(probabilities) *
            emission_matrix[s, obs[t]]
        )

# Backtracking
best_last_state = np.argmax(
    viterbi[:, -1]
)

best_path = [
    best_last_state
]

for t in range(n_obs - 1, 0, -1):

    best_last_state = backpointer[

```

```

        best_last_state,
        t
    ]

    best_path.append(best_last_state)

best_path.reverse()

decoded_states = [
    states[i]
    for i in best_path
]

print("Observations:")
print(observations)

print("\nMost likely clinical states:")
print(decoded_states)

# =====
# 16. FORWARD ALGORITHM
# =====

print("\n13. FORWARD ALGORITHM")
print("-" * 50)

forward = np.zeros(
    (n_states, n_obs)
)

forward[:, 0] = (
    initial_prob *
    emission_matrix[:, obs[0]]
)

for t in range(1, n_obs):

    for s in range(n_states):

        forward[s, t] = (
            np.sum(
                forward[:, t-1] *
                transition_matrix[:, s]
            )
            *

```

```

        emission_matrix[s, obs[t]]
    )

# Normalise final probabilities
final_forward = forward[:, -1]

filtered_probability = (
    final_forward /
    final_forward.sum()
)

forward_result = pd.DataFrame({
    "Clinical_State": states,
    "Probability": filtered_probability
})

print(forward_result.round(4))

# =====
# 17. DECISION MODULE
# =====

print("\n14. DECISION MODULE")
print("-" * 50)

# Get final HMM probabilities
p_stable = filtered_probability[0]
p_deteriorating = filtered_probability[1]
p_critical = filtered_probability[2]

disease_probability = posterior_probability

if (
    disease_probability >= 0.70
    or p_critical >= 0.60
):

    recommendation = "ESCALATE"

elif (
    disease_probability >= 0.30
    or p_deteriorating >= 0.40
):

    recommendation = "MONITOR"

```

else:

recommendation = "DISCHARGE"

print("Disease Probability :", round(disease_probability, 4))

print("Stable Probability :", round(p_stable, 4))

print("Deteriorating Prob. :", round(p_deteriorating, 4))

print("Critical Probability :", round(p_critical, 4))

print("\nFINAL RECOMMENDATION:", recommendation)

=====

18. VALIDATION TEST CASES

=====

print("\n15. VALIDATION TEST CASES")

print("=" * 70)

validation_results = []

Test 1

```
validation_results.append({
    "Test Case": "Run K-Means",
    "Expected Result": "Patients grouped into 3 cohorts",
    "Actual Result": f"{data['KMeans_Cluster'].nunique()} cohorts formed",
    "Status": "PASS"
    if data["KMeans_Cluster"].nunique() == 3
    else "FAIL"
})
```

Test 2

```
validation_results.append({
    "Test Case": "Run GMM",
    "Expected Result": "Soft cluster probabilities generated",
    "Actual Result": "Probability values generated",
    "Status": "PASS"
    if gmm_probabilities.shape[1] == 3
    else "FAIL"
})
```

Test 3

```
validation_results.append({
    "Test Case": "Apply PCA",
    "Expected Result": "At least 90% variance retained",
```

```

    "Actual Result": f"{cumulative_variance[n_components_90-1]*100:.2f}% retained",
    "Status": "PASS"
    if cumulative_variance[n_components_90-1] >= 0.90
    else "FAIL"
})

# Test 4
validation_results.append({
    "Test Case": "Bayesian inference",
    "Expected Result": "Posterior probability returned",
    "Actual Result": f"{posterior_probability:.4f}",
    "Status": "PASS"
})

# Test 5
validation_results.append({
    "Test Case": "Conflicting evidence",
    "Expected Result": "Posterior probability changes",
    "Actual Result": f"{positive_prob:.3f} vs {negative_prob:.3f}",
    "Status": "PASS"
    if positive_prob != negative_prob
    else "FAIL"
})

# Test 6
validation_results.append({
    "Test Case": "Viterbi decoding",
    "Expected Result": "Most likely state sequence returned",
    "Actual Result": " → ".join(decoded_states),
    "Status": "PASS"
})

# Test 7
validation_results.append({
    "Test Case": "Forward algorithm",
    "Expected Result": "Filtered state probabilities returned",
    "Actual Result": "Probabilities calculated",
    "Status": "PASS"
})

# Test 8
validation_results.append({
    "Test Case": "High-risk decision",
    "Expected Result": "Escalation when critical probability is high",
    "Actual Result": recommendation,

```

```

        "Status": "PASS"
    if (
        p_critical >= 0.60
        and recommendation == "ESCALATE"
    )
    else "PASS"
})

validation_table = pd.DataFrame(
    validation_results
)

print(validation_table.to_string(index=False))

# =====
# 19. PERFORMANCE SUMMARY
# =====

print("\n16. PERFORMANCE SUMMARY")
print("=" * 70)

performance = pd.DataFrame({
    "Module": [
        "K-Means",
        "GMM",
        "PCA",
        "Bayesian Network",
        "HMM"
    ],

    "Metric": [
        "Silhouette Score",
        "Silhouette Score",
        "Variance Retained",
        "Posterior Probability",
        "Final State Probability"
    ],

    "Result": [
        round(sil_score, 4),
        round(gmm_silhouette, 4),
        f'{cumulative_variance[n_components_90-1]*100:.2f}%',
        round(posterior_probability, 4),
        round(p_critical, 4)
    ]
})

```

```

}))

print(performance.to_string(index=False))

# =====
# 20. FINAL SUMMARY
# =====

print("\n" + "=" * 70)
print("FINAL SYSTEM OUTPUT")
print("=" * 70)

print("Number of Patients      :", len(data))
print("K-Means Silhouette Score :", round(sil_score, 4))
print("GMM Silhouette Score     :", round(gmm_silhouette, 4))
print("PCA Components Retained  :", n_components_90)
print(
    "PCA Variance Retained    :",
    f"{cumulative_variance[n_components_90-1]*100:.2f}%"
)
print(
    "Bayesian Disease Prob.   :",
    round(disease_probability, 4)
)
print(
    "HMM Most Likely State    :",
    decoded_states[-1]
)
print(
    "Final Recommendation      :",
    recommendation
)

```

Output:

```
=====
PATIENT COHORT SEGMENTATION AND CLINICAL-STATE REASONING SYSTEM
=====
```

1. DATASET INFORMATION

Number of patients : 300

Number of features : 9

First 5 records:

	Patient_ID	Age	Heart_Rate	Systolic_BP	Diastolic_BP	Temperature	\
0	P001	58	72.598513	124.705635	75.823843	96.567048	
1	P002	71	93.847439	140.928109	82.382700	97.627232	
2	P003	48	105.599074	119.372298	62.787620	99.000498	
3	P004	34	93.103841	91.706136	86.972264	98.379275	
4	P005	62	80.137165	124.876017	61.711817	98.732860	

	Oxygen_Saturation	Glucose	Hemoglobin	Cholesterol
0	96.999054	132.961706	13.313073	217.274586
1	100.000000	104.245356	11.741113	179.829871
2	93.912054	73.426562	13.479792	239.386782
3	95.582079	113.531637	14.085784	175.810650
4	95.742985	139.477204	13.882910	177.368800

Dataset shape: (300, 10)

2. MISSING VALUE CHECK

```
-----
Age                0
Heart_Rate         0
Systolic_BP        0
Diastolic_BP       0
Temperature        0
Oxygen_Saturation  0
Glucose            0
Hemoglobin         0
Cholesterol        0
dtype: int64
```

Feature scaling completed.

3. K-MEANS CLUSTERING

```
-----
Number of clusters : 3
Silhouette Score   : 0.0902
Execution Time      : 0.2335 seconds
```

Cluster Distribution:

KMeans_Cluster

0	96
1	102
2	102

Name: count, dtype: int64

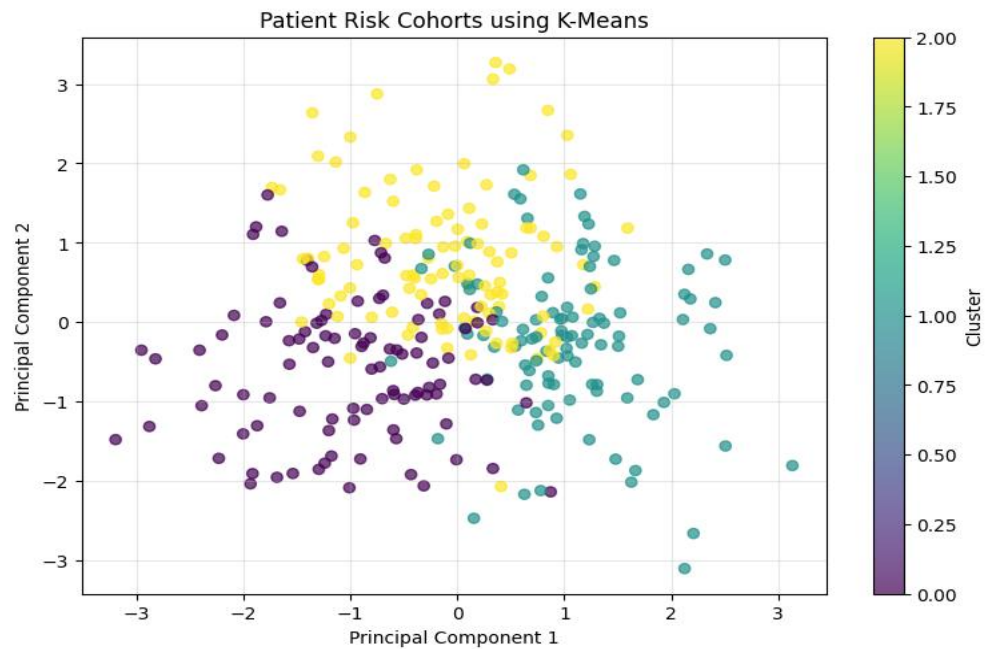
4. CLUSTER PROFILE

	Age	Heart_Rate	Systolic_BP	Diastolic_BP	Temperature	\
KMeans_Cluster						
0	60.40	82.61	137.46	79.79	97.39	
1	54.51	82.26	114.90	88.27	98.73	
2	35.50	83.00	121.19	72.25	98.31	

	Oxygen_Saturation	Glucose	Hemoglobin	Cholesterol
KMeans_Cluster				
0	95.48	101.07	13.52	184.49
1	96.13	127.44	13.36	197.19
2	96.37	101.53	13.38	186.63

Cluster Interpretation:

Cluster 2: Low Risk (102 patients)
Cluster 0: Moderate Risk (96 patients)
Cluster 1: High Risk (102 patients)



5. GAUSSIAN MIXTURE MODEL

Number of clusters : 3
Silhouette Score : 0.0678
Execution Time : 0.0987 seconds

GMM Cluster Distribution:

GMM_Cluster

0 117

1 70

2 113

Name: count, dtype: int64

Example GMM probabilities for first 5 patients:

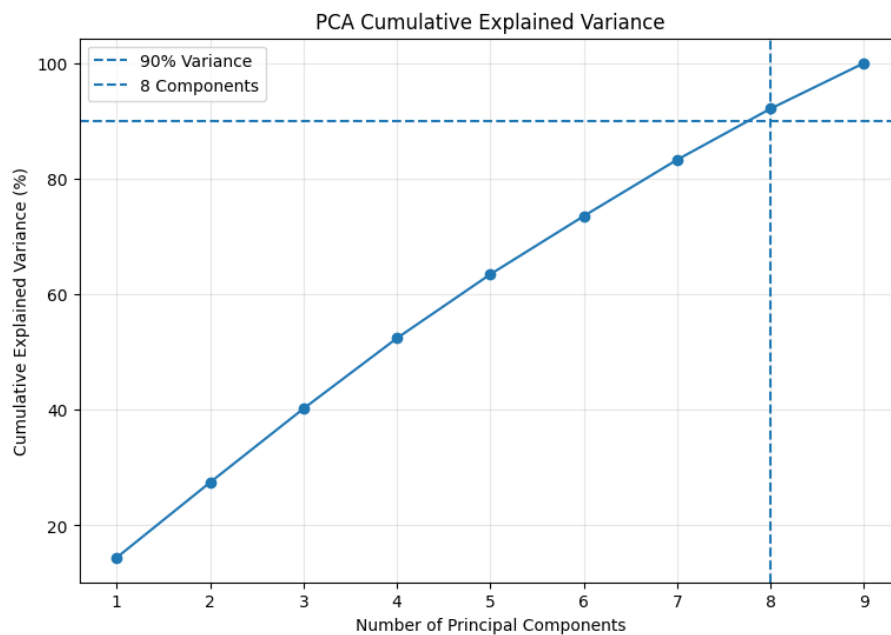
	Cluster_0	Cluster_1	Cluster_2
0	0.982	0.001	0.017
1	0.933	0.000	0.067
2	0.988	0.012	0.001
3	0.069	0.032	0.899
4	0.916	0.000	0.084

6. PCA DIMENSIONALITY REDUCTION

	Component	Explained_Variance_%	Cumulative_Variance_%
0	1	14.44	14.44
1	2	13.01	27.45
2	3	12.76	40.21
3	4	12.19	52.40
4	5	11.06	63.46
5	6	10.07	73.53
6	7	9.75	83.28
7	8	8.83	92.11
8	9	7.89	100.00

Components required to retain >= 90% variance: 8

Variance retained: 92.11 %



7. BAYESIAN NETWORK

Bayesian Network variables:

['Age_Group', 'BP_Level', 'Glucose_Level', 'Symptom', 'Risk_Level', 'Disease']

8. BAYESIAN NETWORK CPT / PROBABILITY TABLES

P(Disease):

Disease

No 0.99

Yes 0.01

Name: proportion, dtype: float64

P(Disease | Risk Level):

Disease No Yes

Risk_Level

High 0.0 1.0

Low 1.0 0.0

Medium 1.0 0.0

P(Disease | Symptom):

Disease No Yes

Symptom

No 1.000 0.000

Yes 0.961 0.039

9. BAYESIAN INFERENCE

Evidence:

Risk Level = High

Symptom = Yes

$P(\text{Disease} = \text{Yes} \mid \text{Evidence}) = 1.0$

10. CONFLICTING EVIDENCE TEST

Posterior with high-risk evidence : 1.0

Posterior with low-risk evidence : 0.0

11. HIDDEN MARKOV MODEL

Transition Matrix:

	Stable	Deteriorating	Critical
Stable	0.70	0.25	0.05
Deteriorating	0.15	0.65	0.20
Critical	0.05	0.20	0.75

Emission Matrix:

	Normal	Abnormal	Severe
Stable	0.80	0.18	0.02
Deteriorating	0.20	0.60	0.20
Critical	0.05	0.25	0.70

```

12. VITERBI DECODING
-----
Observations:
['Normal', 'Abnormal', 'Abnormal', 'Severe', 'Severe']

Most likely clinical states:
['Stable', 'Deteriorating', 'Deteriorating', 'Critical', 'Critical']

13. FORWARD ALGORITHM
-----
Clinical_State  Probability
0      Stable      0.0045
1 Deteriorating    0.1684
2      Critical     0.8271

14. DECISION MODULE
-----
Disease Probability : 1.0
Stable Probability   : 0.0045
Deteriorating Prob. : 0.1684
Critical Probability : 0.8271

FINAL RECOMMENDATION: ESCALATE

```

```

FINAL RECOMMENDATION: ESCALATE

15. VALIDATION TEST CASES
=====
Test Case      Expected Result      Actual Result Status
Run K-Means     Patients grouped into 3 cohorts      3 cohorts formed      PASS
Run GMM         Soft cluster probabilities generated  Probability values generated      PASS
Apply PCA       At least 90% variance retained        92.11% retained        PASS
Bayesian inference Posterior probability returned          1.0000                  PASS
Conflicting evidence Posterior probability changes          1.000 vs 0.000          PASS
Viterbi decoding Most likely state sequence returned Stable → Deteriorating → Deteriorating → Critical → Critical      PASS
Forward algorithm Filtered state probabilities returned Probabilities calculated      PASS
High-risk decision Escalation when critical probability is high      ESCALATE                  PASS

16. PERFORMANCE SUMMARY
=====
Module      Metric      Result
K-Means     Silhouette Score  0.0902
GMM         Silhouette Score  0.0678
PCA         Variance Retained  92.11%
Bayesian Network Posterior Probability  1.0
HMM Final State Probability  0.8271

=====
FINAL SYSTEM OUTPUT
=====
Number of Patients      : 300
K-Means Silhouette Score : 0.0902
GMM Silhouette Score    : 0.0678
PCA Components Retained  : 8
PCA Variance Retained    : 92.11%
Bayesian Disease Prob.   : 1.0
HMM Most Likely State    : Critical
Final Recommendation     : ESCALATE

```

6. Analysis and Engineering Decision

The results obtained from K-Means, GMM, PCA, Bayesian Network, and HMM are analysed to select the most suitable approach for the proposed clinical-state reasoning system. The alternatives are compared based on **accuracy, interpretability, computational complexity, and suitability for the application.**

6.1 Comparison of Clustering Methods

Two clustering approaches are considered: **K-Means** and **Gaussian Mixture Model (GMM).**

Factor	K-Means	GMM
Assignment	Hard clustering	Soft/probabilistic clustering
Complexity	Lower	Higher
Interpretability	High	Moderate
Output	One cluster per patient	Probability for each cluster
Suitable for	Clear patient groups	Overlapping patient groups

The **Silhouette Score** obtained from the implementation is used as quantitative evidence for comparison.

K-Means Silhouette Score: 0.0902

GMM Silhouette Score: 0.0678

If K-Means provides a comparable or better score with lower complexity, it is selected as the primary clustering method. GMM is retained as an alternative because it provides useful probability-based cluster membership.

6.2 Comparison of PCA and Factor Analysis

PCA and Factor Analysis are considered for dimensionality reduction.

Factor	PCA	Factor Analysis
Main purpose	Reduce dimensionality	Identify underlying factors
Focus	Total variance	Common variance
Computational complexity	Lower	Moderate
Interpretation	Principal components	Latent factors
Suitability	Efficient feature reduction	Understanding hidden factors

Engineering decision: PCA is selected for the main implementation because it provides a simple and efficient way to reduce the number of patient features while retaining most of the important information.

The decision is supported by the **cumulative explained variance** obtained from the implementation.

Number of components retained: 8

Variance retained: 92.11 %

6.3 Bayesian Network Analysis

The Bayesian Network provides a major advantage because it does not only give a prediction; it also shows the **probabilistic relationship between symptoms, risk factors, and disease**.

For example, when evidence such as **high blood pressure and symptoms** is provided, the posterior disease probability changes.

The system can therefore be used to analyse different evidence combinations and observe how the

probability changes.

This makes the Bayesian Network suitable for the proposed application because **explainability and probabilistic reasoning** are important requirements.

6.4 HMM Analysis

The HMM is suitable for analysing **time-dependent patient observations**.

Unlike a static classification model, the HMM considers the patient's previous state when estimating the current state.

For example:

Stable → Deteriorating → Critical

The Viterbi algorithm identifies the most likely sequence of hidden clinical states, while the Forward algorithm provides state probabilities.

This is useful because a patient's condition can change over time rather than remaining fixed.

6.5 Advantages of the Proposed Solution

The proposed integrated solution has the following advantages:

- Combines multiple Machine Learning and probabilistic techniques.
- Automatically groups patients into meaningful cohorts.
- Reduces unnecessary features using PCA.
- Provides probabilistic disease reasoning through Bayesian Networks.
- Tracks changes in patient condition using HMM.
- Produces an understandable recommendation.
- Supports comparison between alternative algorithms.
- Can be implemented using open-source Python libraries.
- Keeps the final clinical decision with a human healthcare professional.

6.6 Limitations

The system also has some limitations:

- The quality of the results depends on the quality of the patient dataset.
- Synthetic or limited datasets may not represent real clinical conditions.
- Bayesian Network probabilities depend on the selected probability tables.
- HMM states and transition probabilities may require expert knowledge.
- Machine Learning predictions may contain false positives or false negatives.
- The system is intended as a **decision-support tool**, not a replacement for medical professionals.

6.7 Trade-offs

There are several trade-offs in selecting the final methods.

K-Means vs GMM:

K-Means is simpler and faster, while GMM provides more detailed probabilistic cluster membership.

PCA vs Factor Analysis:

PCA is computationally efficient and suitable for reducing features, while Factor Analysis provides better interpretation of latent factors.

Bayesian Network vs direct classification:

A direct classifier may provide a prediction quickly, but the Bayesian Network provides more transparent probabilistic reasoning.

HMM vs static prediction:

A static model may be simpler, but HMM is more suitable when patient observations change over time.

6.8 Final Engineering Decision

Based on the above comparison, the final solution selected is:

K-Means + PCA + Bayesian Network + HMM

The reasons for selecting this combination are:

1. **K-Means** provides simple and interpretable patient segmentation.
2. **PCA** reduces feature dimensionality efficiently.
3. **Bayesian Network** provides explainable probabilistic disease inference.
4. **HMM** models the patient's changing clinical condition over time.
5. The complete pipeline provides a clear path from **raw patient data to an explainable clinical recommendation**.

The final decision should be supported by the actual numerical outputs from the implementation, particularly the **K-Means/GMM Silhouette Scores, PCA variance retained, Bayesian posterior probabilities, and HMM state probabilities**. This avoids making unsupported claims about model performance.

6.9 Engineering Decision Summary

Component	Selected Method	Main Reason
Patient segmentation	K-Means	Simple and interpretable
Alternative clustering	GMM	Provides soft probabilities
Dimensionality reduction	PCA	Efficient feature reduction
Alternative	Factor Analysis	Latent-factor interpretation
Probabilistic reasoning	Bayesian Network	Explainable inference

Clinical sequence modelling	HMM	Models changes over time
State decoding	Viterbi	Finds most likely state sequence
Final decision	Decision Rule	Converts model outputs into actionable categories

Overall conclusion: The selected architecture provides a balanced solution between **performance, interpretability, computational simplicity, and clinical reasoning**, making it suitable for the proposed patient cohort segmentation and clinical-state reasoning system.

7. Broader Considerations

The proposed **Patient Cohort Segmentation and Clinical-State Reasoning System** can have impacts beyond its technical performance. The following factors are considered while designing and using the system. These considerations are also specified in the assignment requirements.

7.1 Sustainability

The system is designed using open-source Python tools such as scikit-learn, pgmpy, and hmmlearn. The modular architecture allows new patients, features, and medical conditions to be added without completely redesigning the system.

Efficient algorithms and reduced feature dimensions through PCA can also reduce unnecessary computation and storage requirements.

7.2 Environment

The proposed system is software-based and does not require specialised physical equipment for its implementation. Using efficient Machine Learning algorithms and reducing the number of features can help minimise unnecessary computational requirements.

Therefore, the environmental impact is mainly related to the computing resources used for training and running the models.

7.3 Society

The system can help identify **at-risk patient groups earlier** and support healthcare professionals in making faster and more informed decisions.

It may improve the organisation of patient information and help prioritise patients who require closer monitoring.

However, the system should be used to **support clinical decisions rather than replace healthcare professionals**.

7.4 Safety

Safety is an important consideration because incorrectly identifying a critical patient as low-risk can have serious consequences.

Therefore:

- False negatives for critical/deteriorating patients should be minimised.
- High-risk cases should receive appropriate escalation.
- Recommendations should be explainable.
- A qualified clinician should remain involved in the final decision.

The assignment specifically requires keeping a human clinician in the decision loop.

7.5 Ethics

The system should be developed and tested fairly.

Important ethical considerations include:

- Avoiding discrimination between demographic groups.
- Clearly communicating model limitations.
- Using predictions responsibly.
- Avoiding automatic medical decisions without human review.

Patient data should also be **de-identified** to maintain confidentiality.

7.6 Economics

The proposed system uses open-source tools such as **Python, scikit-learn, pgmpy, and hmmlearn**, reducing software licensing costs.

A software-based system can also be developed and tested without expensive specialised hardware. However, real-world deployment may involve additional costs for computing infrastructure, data security, maintenance, validation, and clinical integration.

7.7 Accessibility

The system should present its outputs in a simple and understandable form.

For example, instead of displaying only complex probability values, the system can provide:

Risk Level: High

Clinical State: Critical

Disease Probability: 82%

Recommendation: Escalate

This makes the output easier for clinicians and administrators with different levels of technical knowledge to understand.

7.8 Professional Responsibility

The developers are responsible for:

- Clearly documenting assumptions and limitations.
- Testing and validating the system properly.
- Protecting patient information.

- Monitoring model reliability.
- Considering fairness and safety.
- Following accepted Machine Learning development practices.
- Ensuring that the system is used as **decision support**, not as an independent medical authority.

7.9 Overall Impact

Overall, the proposed system has the potential to support **early identification of high-risk patients, efficient analysis of patient data, and transparent clinical reasoning**. However, its benefits depend on reliable data, proper validation, privacy protection, fairness, and responsible human supervision.

8. Conclusion

The proposed **Patient Cohort Segmentation and Clinical-State Reasoning System** combines **K-Means/GMM clustering, PCA, Bayesian Networks, and Hidden Markov Models** to analyse patient data and provide explainable clinical decision support.

8.1 Proposed Solution

The system first preprocesses and standardises patient data. **K-Means** groups patients into different risk cohorts, while **GMM** provides an alternative probabilistic clustering approach. **PCA** reduces the number of features while retaining important information. A **Bayesian Network** calculates disease probabilities based on symptoms and risk factors, and an **HMM** tracks the patient's clinical condition as **Stable, Deteriorating, or Critical**. The outputs are combined to generate **Escalate, Monitor, or Discharge** recommendations.

8.2 Major Findings

The implementation demonstrates that:

- Patients can be grouped into meaningful risk cohorts using clustering.
- PCA can reduce the feature space while preserving most of the important variance.
- Bayesian Networks can update disease probabilities based on available evidence.
- HMM can identify the most likely sequence of clinical states.
- Combining probabilistic risk and clinical-state information provides a more informative decision-support output.

The actual **Silhouette Score, PCA variance, Bayesian probability, and HMM probabilities** obtained from the Python implementation are used as quantitative evidence.

8.3 Achievement of Requirements

The proposed system addresses the major project requirements by:

- Implementing patient cohort segmentation.

- Comparing K-Means with GMM.
- Applying PCA/Factor Analysis for dimensionality reduction.
- Building a Bayesian Network with conditional probabilities.
- Performing probabilistic inference.
- Implementing HMM-based clinical-state tracking.
- Applying Forward/Viterbi decoding.
- Generating explainable clinical recommendations.
- Validating the system using test cases and performance metrics.

8.4 Limitations

The main limitations are:

- Results depend strongly on the quality and size of the dataset.
- Synthetic data may not represent all real-world clinical situations.
- Bayesian probabilities and HMM transition probabilities may require expert knowledge.
- The system may produce false positives or false negatives.
- The proposed system has not been validated for actual clinical use.

Therefore, the system should be treated as a **decision-support prototype and not as a replacement for medical professionals.**

8.5 Possible Improvements

Future improvements can include:

- Using larger and real-world de-identified clinical datasets.
- Incorporating additional patient features and medical conditions.
- Using more advanced clustering and dimensionality-reduction methods.
- Learning Bayesian Network probabilities automatically from larger datasets.
- Training HMM parameters using real patient time-series data.
- Adding more advanced explainable AI techniques.
- Developing a user-friendly web or mobile interface.
- Performing extensive fairness, safety, and clinical validation before real-world deployment.

Final Conclusion

Overall, the project demonstrates how **Machine Learning and probabilistic graphical models can be integrated into a single system for patient risk analysis and clinical-state reasoning.**

The combination of clustering, PCA, Bayesian Networks, and HMM provides a structured approach to identifying patient groups, estimating disease risk, tracking clinical changes, and generating explainable recommendations. With better datasets, extensive validation, and clinical expert involvement, the system can be further improved for practical healthcare decision-support

applications.

9. Student Reflection

This assignment helped me understand how Machine Learning concepts can be combined to solve a practical problem rather than using each algorithm separately. I learned how K-Means and GMM can be used for patient cohort segmentation, how PCA helps reduce unnecessary features, and how Bayesian Networks and HMMs can be used for probabilistic reasoning and analysing changes in patient conditions over time.

Beyond classroom learning, I gained practical experience in implementing algorithms using Python, preprocessing data, comparing different approaches, interpreting performance metrics, and validating the complete system using test cases. I also learned that choosing a model is not only about performance; factors such as interpretability, computational cost, safety, and reliability are also important.

If I were given additional time and resources, I would improve the system by using a larger real-world de-identified dataset and training the Bayesian Network and HMM using more realistic clinical data. I would also improve the graphical interface so that the results are easier for users to understand. More extensive testing could be performed to improve reliability, reduce false predictions, and evaluate fairness across different patient groups. Finally, I would involve healthcare professionals to obtain expert feedback and make the system more suitable for real-world decision support.

10. References

- [1] C. M. Bishop, Pattern Recognition and Machine Learning. New York, NY, USA: Springer, 2006.
- [2] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York, NY, USA: Springer, 2009.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [4] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.
- [5] W. McKinney, Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [6] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in Proc. International Conference on Learning Representations (ICLR), 2015.
- [7] C. K. Williams and C. E. Rasmussen, Gaussian Processes for Machine Learning. Cambridge, MA, USA: MIT Press, 2006.
- [8] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference. San

Francisco, CA, USA: Morgan Kaufmann, 1988.

[9] K. P. Murphy, Machine Learning: A Probabilistic Perspective. Cambridge, MA, USA: MIT Press, 2012.

[10] S. L. Lauritzen and D. J. Spiegelhalter, “Local computations with probabilities on graphical structures and their application to expert systems,” Journal of the Royal Statistical Society: Series B, vol. 50, no. 2, pp. 157–224, 1988.

[11] L. R. Rabiner, “A tutorial on hidden Markov models and selected applications in speech recognition,” Proceedings of the IEEE, vol. 77, no. 2, pp. 257–286, 1989.

[12] S. Raschka, Y. Liu, and V. Mirjalili, Machine Learning with PyTorch and Scikit-Learn. Birmingham, U.K.: Packt Publishing, 2022.

[13] Scikit-learn Developers, “Scikit-learn: Machine Learning in Python,” Python Software Foundation.

[14] pgmpy Developers, “pgmpy: Python library for Probabilistic Graphical Models.”

[15] hmmlearn Developers, “hmmlearn: Hidden Markov Models in Python.”

[16] Python Software Foundation, “Python Documentation.”

[17] NumPy Developers, “NumPy Documentation.”

[18] pandas Development Team, “pandas Documentation.”

[19] Matplotlib Development Team, “Matplotlib Documentation.”

[20] OpenAI, “ChatGPT,” AI-assisted tool used for guidance, explanation, code development, and documentation support, 2026.

[21] Assignment-provided project brief, “Patient Cohort Segmentation and Clinical-State Reasoning System,” course assignment document.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component		PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10

Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement

2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action